

SET9

Ким София Денисовна

Отчет по заданию A1

Посылки

A1m: [375345348](#)

A1q: [375359318](#)

A1r: [375347602](#)

A1rq: [375359423](#)

Файл с результатами: a1_all_results.csv

Публичный репозиторий с исходным кодом и результатами эмпирических замеров:
https://github.com/sskimdp/SET9_a1_KimSD

Реализованные алгоритмы

- стандартный MergeSort;
- стандартный QuickSort;
- StringMergeSort с использованием LCP-сравнений;
- тернарный StringQuickSort;
- MSD Radix Sort без переключения;
- MSD Radix Sort с переключением на StringQuickSort при размере подмассива меньше 74.

Для стандартного QuickSort и специализированного StringQuickSort способ выбора опорного элемента сделан одинаковым: в обоих случаях используется элемент из середины текущего подмассива.

StringMergeSort реализован по схеме LCP-сравнения: при сравнении двух строк используется известная длина общего префикса, поэтому уже проверенные символы не сравниваются повторно. В процессе слияния хранятся пары вида (строка, длина LCP с предыдущей строкой).

Генерация тестовых данных

Для подготовки тестовых данных был реализован класс StringGenerator. Он формирует строки длиной от 10 до 200 символов над заданным алфавитом из 74 символов. В эксперименте использовались массивы размеров от 100 до 3000 с шагом 100: случайные, обратно отсортированные, почти отсортированные, а также массивы со строками, имеющими общие префиксы. Для каждого типа данных сначала генерировался массив максимального размера 3000, после чего из него брались подмассивы нужной длины.

Методика эксперимента

Для проведения экспериментов реализован отдельный класс StringSortTester. Для каждого алгоритма и каждого набора данных исходный массив копировался, после чего

выполнялась сортировка. Время измерялось с помощью `high_resolution_clock`, а результат усреднялся по 5 запускам.

Для сравнительных алгоритмов учитывалось количество посимвольных сравнений. Для MSD Radix Sort и его модификации учитывалось количество посимвольных обращений при распределении строк по корзинам, так как эти алгоритмы не выполняют прямого сравнения строк между собой. В итоговой таблице эта величина обозначена как `char_operations`.

Сводные результаты для размера 3000

В таблице ниже приведены результаты для максимального размера массива

Тип массива	Алгоритм	Время, мкс	Посимвольные операции
Almost sorted	StringQuickSort	145.47	174089
Almost sorted	MsdRadixQuickSort	180.44	35691
Almost sorted	QuickSort	279.27	452960
Almost sorted	MsdRadixSort	378.41	14604
Almost sorted	MergeSort	915.15	40773
Almost sorted	StringMergeSort	2457.56	15752
Common prefixes	StringQuickSort	90.54	76730
Common prefixes	MsdRadixQuickSort	447.62	44981
Common prefixes	QuickSort	495.62	622663
Common prefixes	MsdRadixSort	675.61	43822
Common prefixes	MergeSort	1243.37	154085
Common prefixes	StringMergeSort	2627.11	39928
Random	StringQuickSort	90.26	53225
Random	MsdRadixQuickSort	204.83	27624

Random	MsdRadixSort	411.07	14550
Random	QuickSort	452.97	478015
Random	MergeSort	1137.68	44545
Random	StringMergeSort	2661.58	28042
Reverse sorted	StringQuickSort	176.85	147626
Reverse sorted	MsdRadixQuickSort	212.01	55078
Reverse sorted	QuickSort	274.13	453426
Reverse sorted	MsdRadixSort	444.52	14502
Reverse sorted	MergeSort	819.89	25816
Reverse sorted	StringMergeSort	2395.27	11118

Графики времени работы

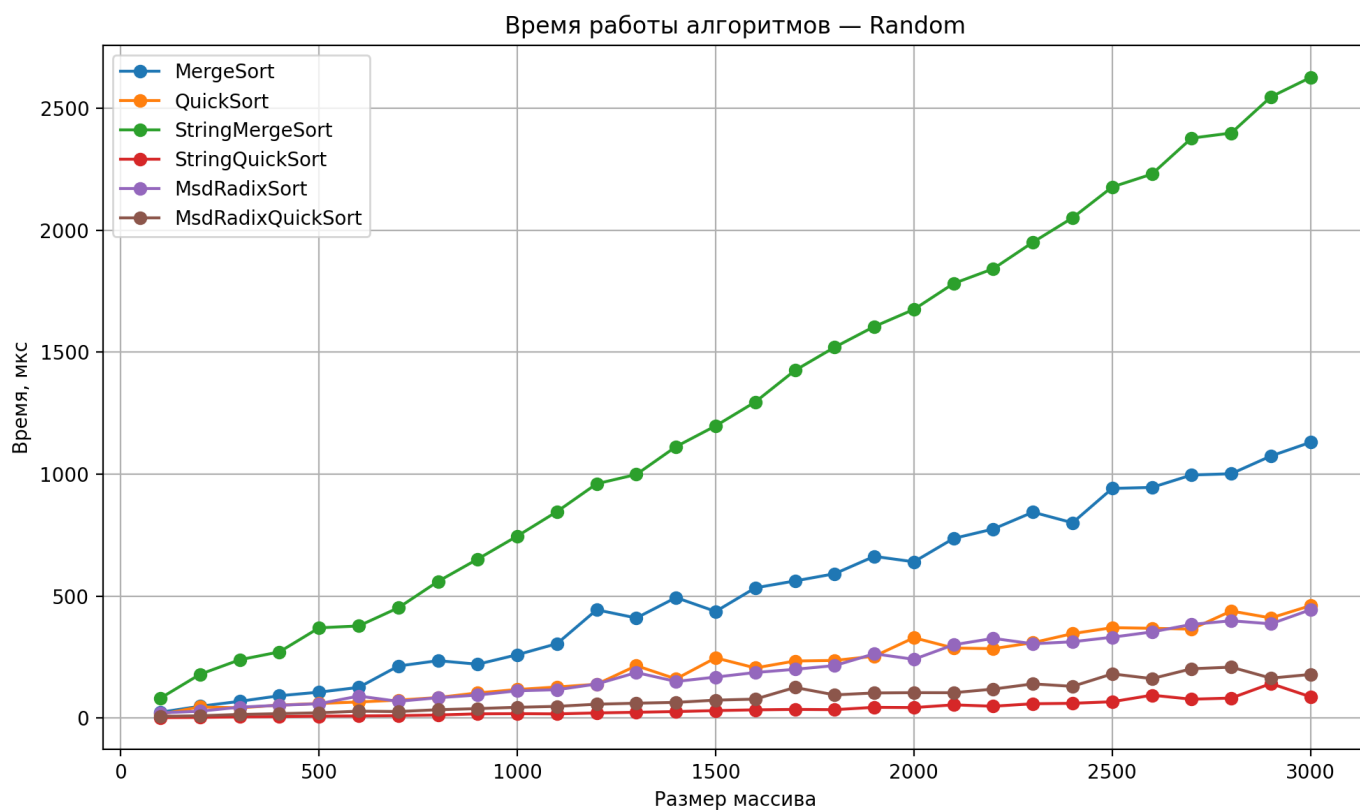


Рисунок 1 – Время работы алгоритмов: случайные массивы

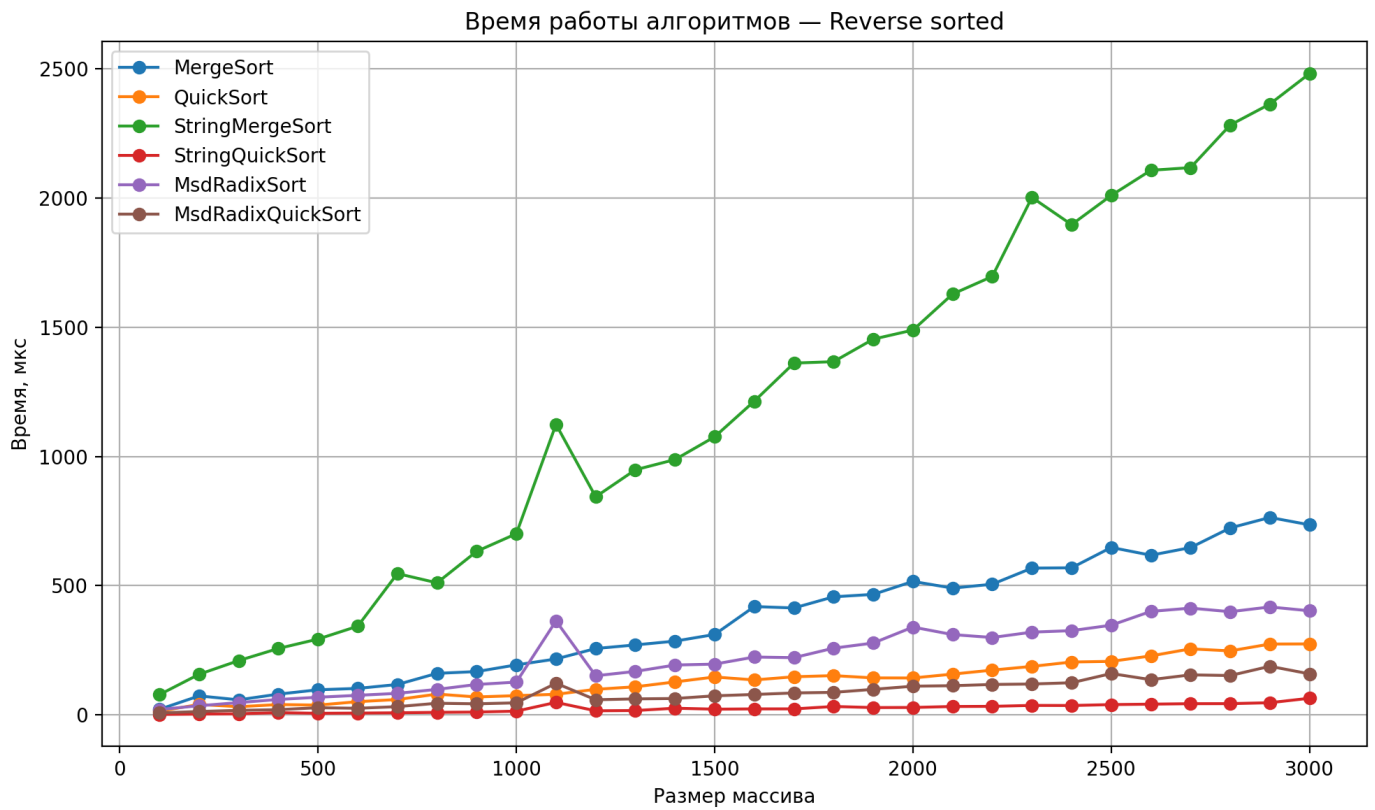


Рисунок 2 – Время работы алгоритмов: обратно отсортированные массивы

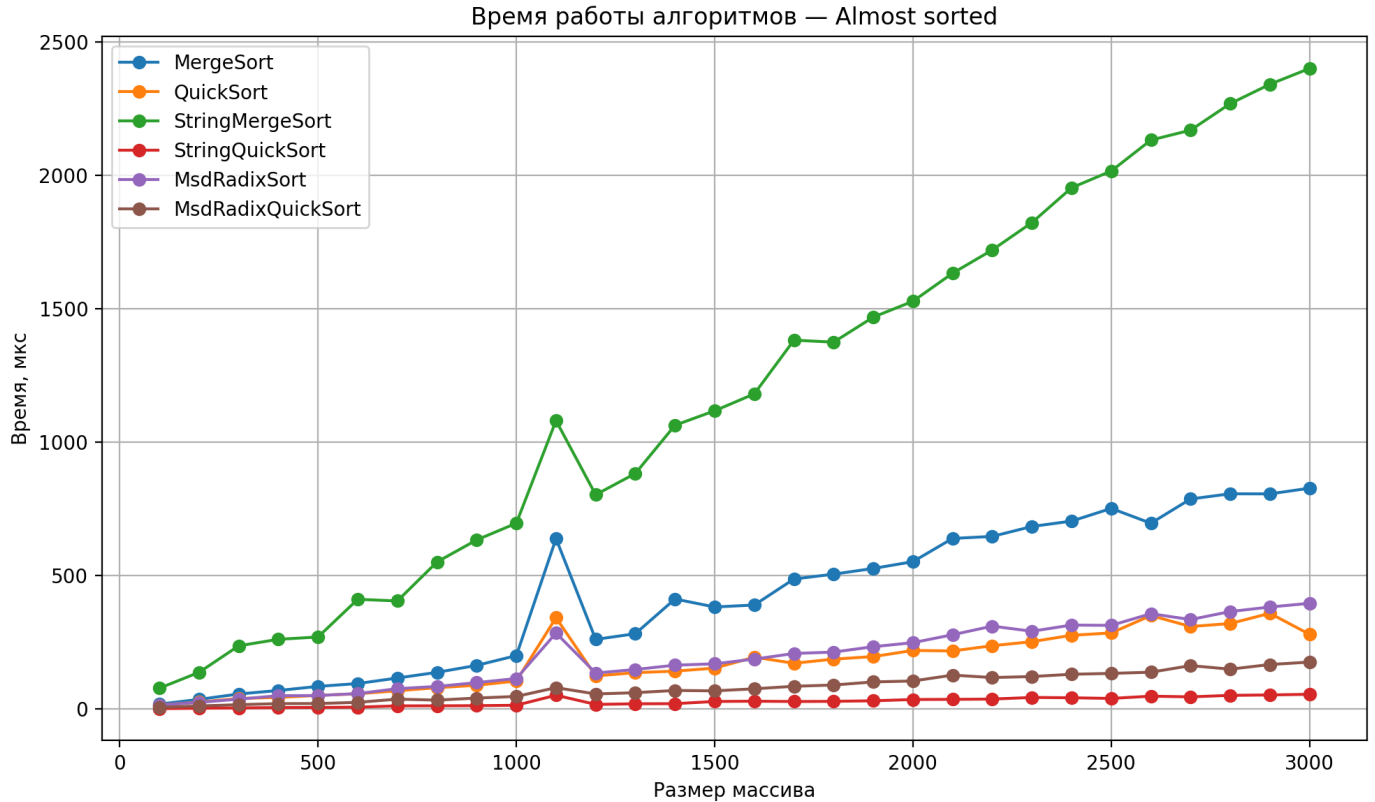


Рисунок 3 – Время работы алгоритмов: почти отсортированные массивы

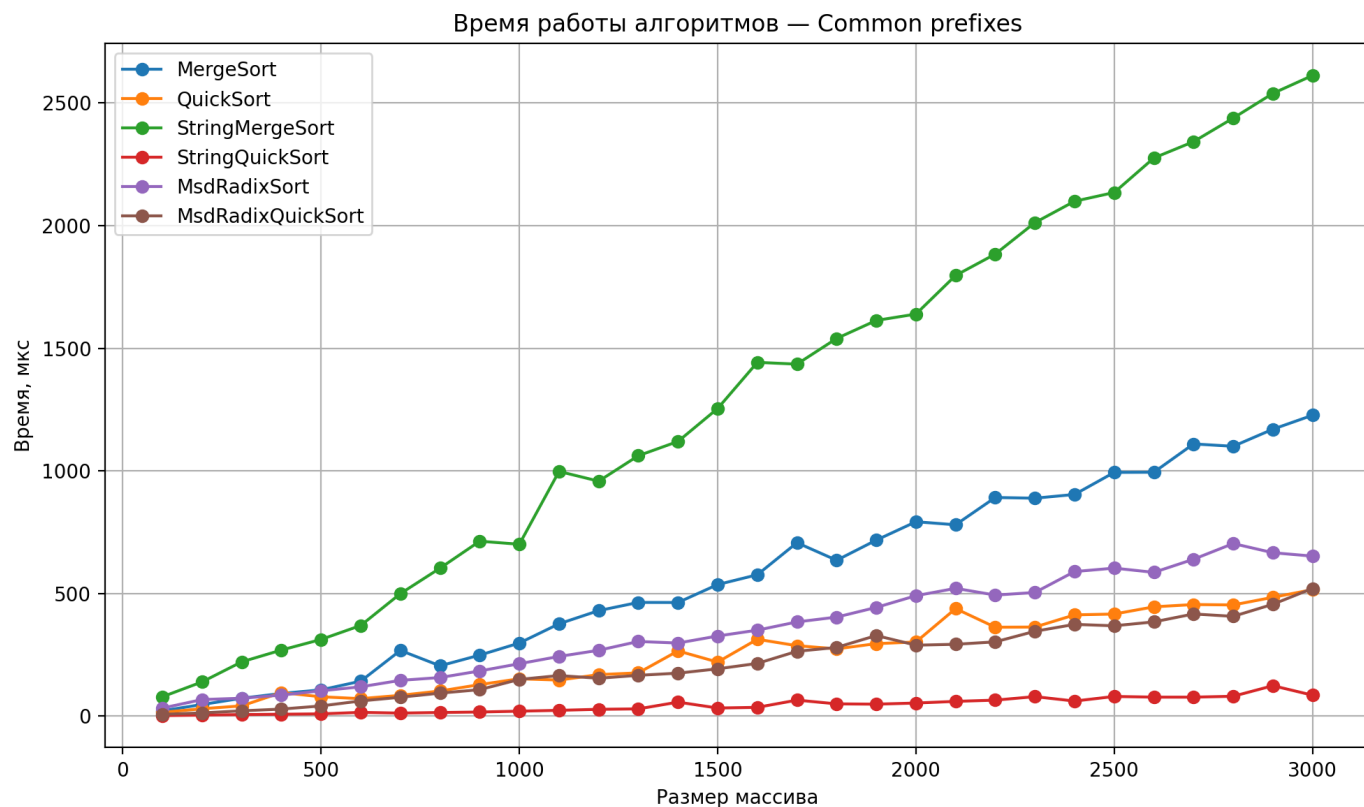


Рисунок 4 – Время работы алгоритмов: массивы с общими префиксами

Графики посимвольных операций

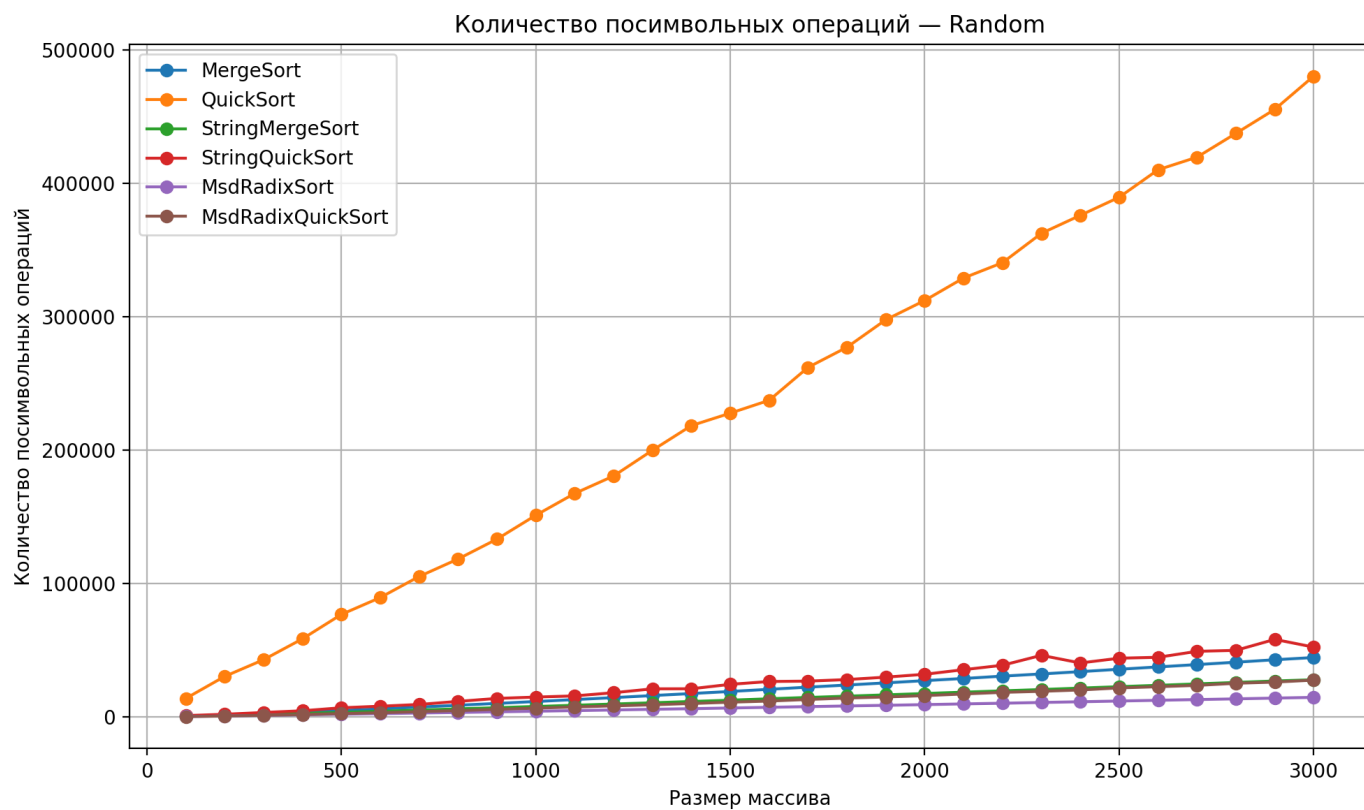


Рисунок 5 – Посимвольные операции: случайные массивы

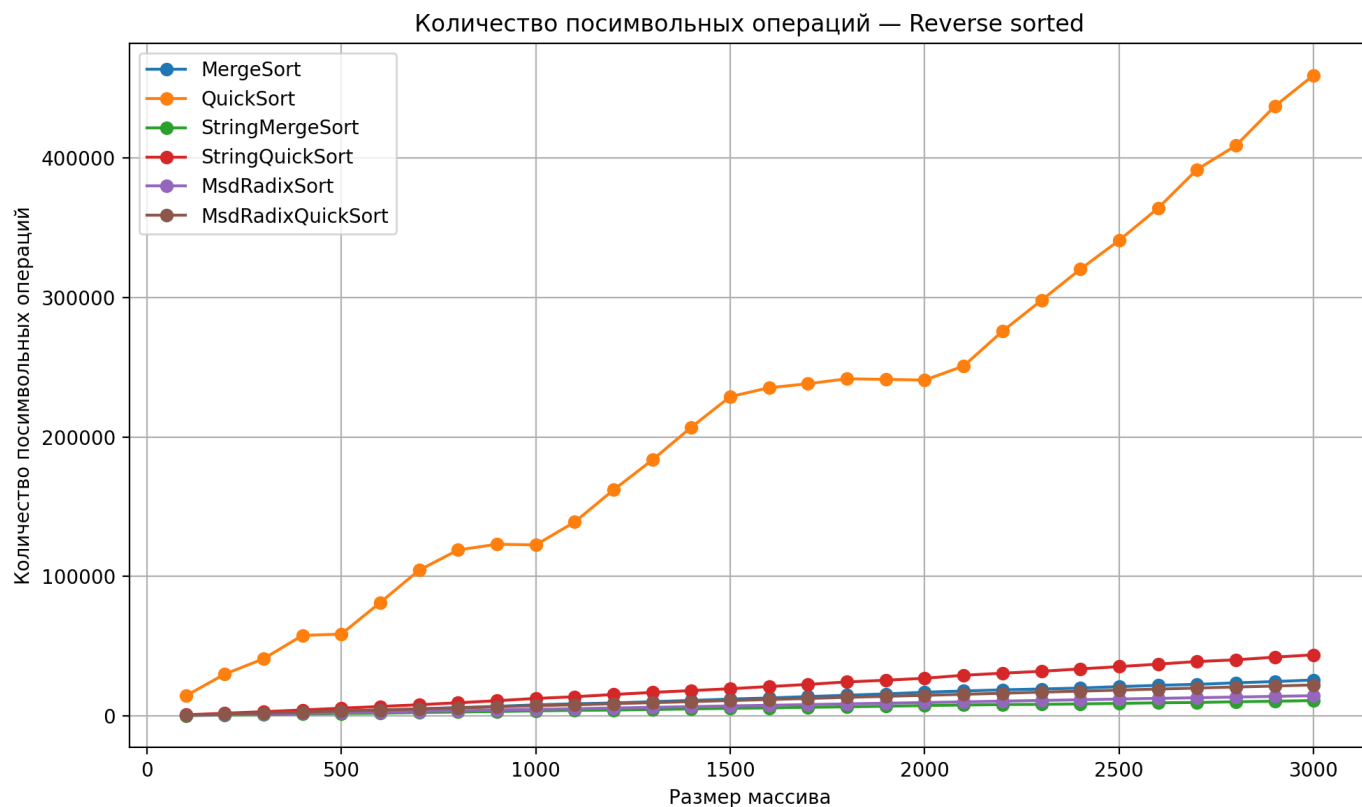


Рисунок 6 – Посимвольные операции: обратно отсортированные массивы

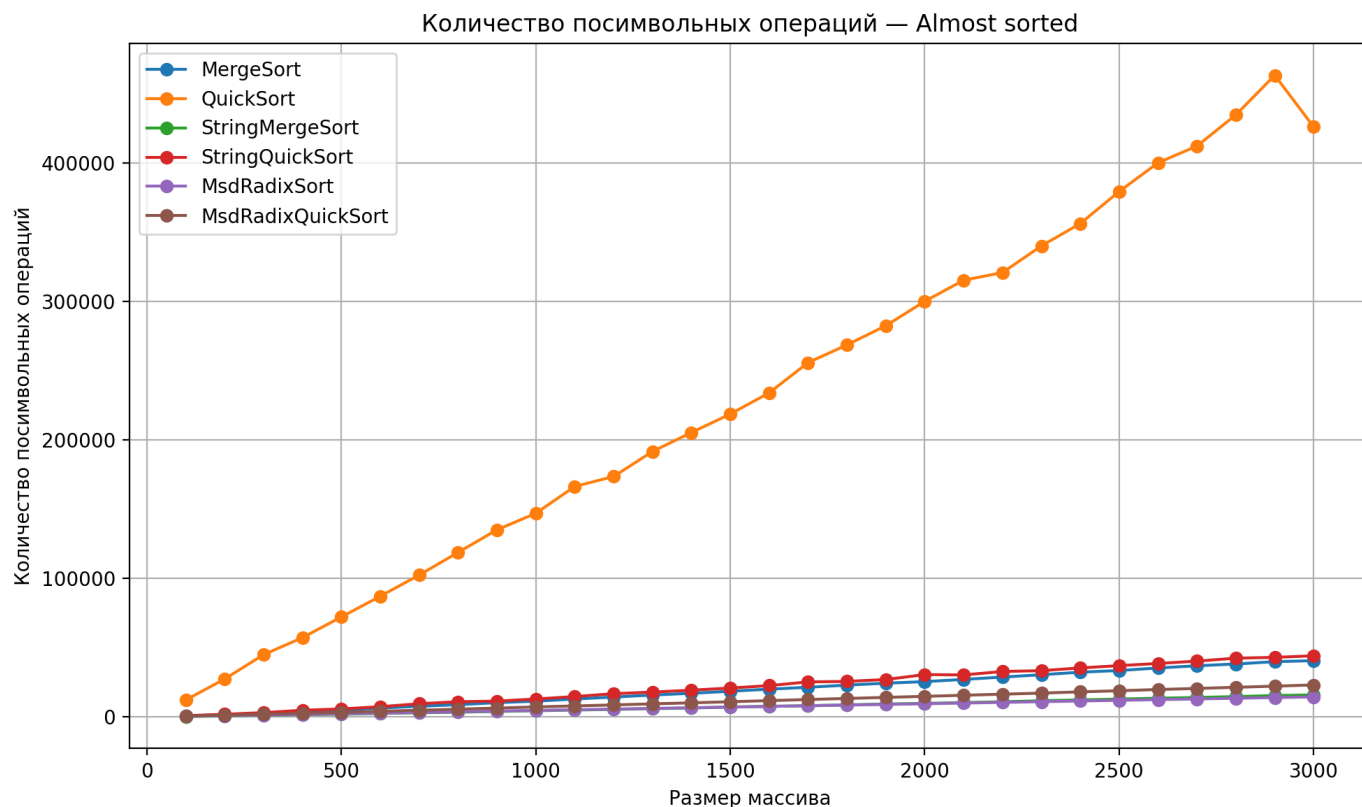


Рисунок 7 – Посимвольные операции: почти отсортированные массивы

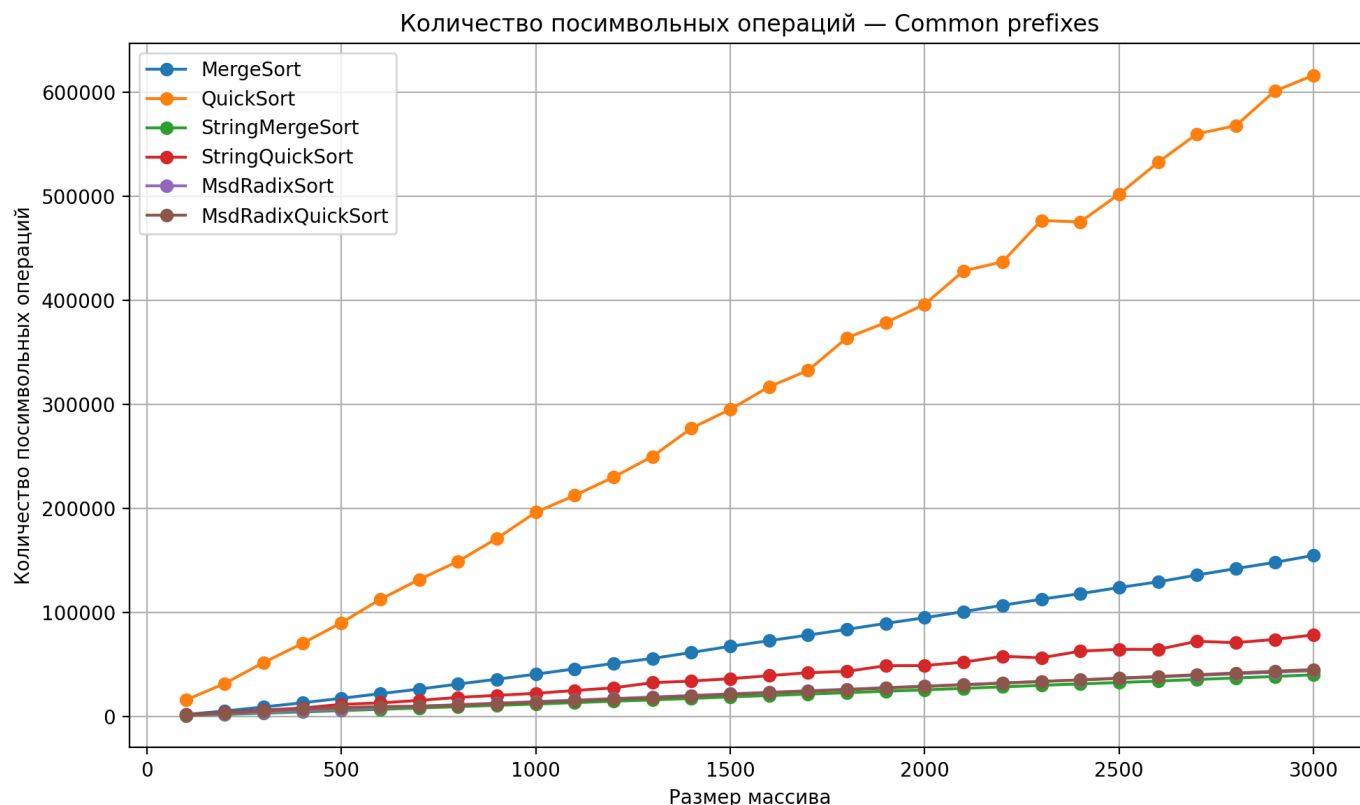


Рисунок 8 – Посимвольные операции: массивы с общими префиксами

Анализ результатов

На всех типах данных специализированный StringQuickSort показывает наименьшее или одно из наименьших значений времени работы. Это объясняется тем, что алгоритм использует структуру строк: он сортирует элементы по символам на текущей глубине и не выполняет полноценное сравнение строк при каждом шаге. На случайных данных при размере 3000 строк StringQuickSort оказался быстрее стандартного QuickSort и заметно быстрее стандартного MergeSort.

Стандартный QuickSort часто показывает хорошее время, однако количество посимвольных операций у него значительно больше, особенно на массивах с общими префиксами. Это связано с тем, что обычное сравнение строк повторно проходит одинаковые начальные фрагменты строк при разных сравнениях. В эксперименте для массива Common prefixes размера 3000 у QuickSort зафиксировано 622663 посимвольные операции, тогда как у StringQuickSort – 76730.

MSD Radix Sort демонстрирует малое число посимвольных операций, поскольку каждая строка распределяется по корзинам по символам. Однако по времени он может проигрывать StringQuickSort из-за накладных расходов на массивы подсчёта и копирование элементов. Модификация MsdRadixQuickSort уменьшает эти накладные расходы на малых подмассивах за счёт переключения на StringQuickSort, поэтому по времени она обычно лучше чистого MSD Radix Sort.

StringMergeSort использует LCP-сравнения и действительно уменьшает число повторных посимвольных сравнений. При этом по времени в данной реализации он оказался медленнее других адаптированных алгоритмов. Это связано с дополнительными затратами

на хранение пар со значениями LCP, рекурсивное формирование промежуточных векторов и копирование данных при слиянии. Таким образом, теоретическое преимущество по числу сравнений не всегда напрямую превращается в выигрыш по фактическому времени работы.

Сравнение с теоретическими оценками

Алгоритм	Теоретическая оценка	Соответствие эксперименту
MergeSort	$O(n \log n)$ сравнений строк	Время растёт плавно с увеличением размера массива; на строках затраты зависят от длины общих префиксов.
QuickSort	В среднем $O(n \log n)$ сравнений строк	По времени быстрее MergeSort, но число посимвольных сравнений велико, особенно при общих префиксах.
StringMergeSort	$O(\sum \text{LCP}(R) + n \log n)$	Число операций ниже, чем у обычных сравнительных сортировок, но время увеличено из-за накладных расходов реализации.
StringQuickSort	Близко к $O(n \log n)$ с использованием посимвольного разбиения	Показывает лучший результат по времени на большинстве наборов данных.
MSD Radix Sort	Зависит от количества обработанных символов; близко к $O(W)$, где W — суммарная длина обработанных префиксов	Число операций мало, но время увеличивается из-за распределения по корзинам и копирования.
MsdRadixQuickSort	MSD Radix Sort + StringQuickSort на малых подмассивах	Обычно быстрее чистого MSD Radix Sort за счёт уменьшения накладных расходов на малых группах.